

# MPI: Derived Data Types

- any data layout can be described with them
- derived from basic MPI data types
  - for passing data organized as AoS
- allow for efficient transfer of non-contiguous and heterogeneous data
  - example: halo exchange
  - during communication MPI data type tells MPI system where to get the data and where to put it
- both solutions help user to avoid hand-coding
- libraries should have efficient implementations
  - more general data types are slower
  - no need for MPI\_Pack and MPI\_Unpack
  - overhead is reduced as only one long message is sent

## Derived Data type

- an object used to describe a data layout in memory by
  - a sequence of basic data types
  - a sequence of displacements
- constructed and destroyed during runtime
- structure
  - Typemap: pairs of basic MPI data types and displacements
    - {(type 0, displacement 0), ..., (type N-1, displacement N-1)}
    - example: {(int, 0), (double, 8), (char, 16)}
- data type routines
  - construction
    - MPI\_Type\_contiguous: contiguous data type
    - MPI\_Type\_vector: regularly spaced data type
    - MPI\_Type\_indexed: variably spaced data type
    - MPI\_Type\_create\_subarray: describes subarray of an array
    - MPI\_Type\_create\_struct: general data type
  - commit
    - MPI\_Type\_commit: must be called before new data type can be used
  - free
    - MPI\_Type\_free: marks a data type for deallocation

### MPI\_Type\_contiguous

- output data type is obtained by concatenating defined number of copies of input data type
- constructs a typemap for output data type consisting of replications of input data type
- example: matrix row as a data type

### MPI\_Type\_vector

- similar to MPI\_Type\_continuous but with self-defined stride
- input
  - number of blocks
  - number of elements of input data type in each block
  - stride - number of elements between beginnings of neighbouring blocks
  - example: matrix column as a data type

### MPI\_Type\_indexed

- generalization of MPI\_Type\_vector
  - number of blocks
  - for each block we specify number of elements and stride

### MPI\_Type\_create\_subarray

- creates a data type which is a sub-array of an array
- useful for column-wise distribution of data

### MPI\_Type\_create\_struct

- Typemap
  - pairs of basic data types and displacements
- extent
  - span from the lower to the upper bound
  - inner holes are counted, holes at the end are not!
  - important for alignment of data types to data, not for construction and memory allocation
  - query: MPI\_Type\_get\_extent
- size
- number of bytes that has to be transferred
- holes are not counted
- query: MPI\_Type\_size
- Example
  - to get displacements
  - MPI\_Type\_get\_extent
  - MPI\_Get\_address

### MPI\_Type\_create\_resized

- output data type is identical to the input data type but lower bound and extent are changed
- useful to correct stride for communication
  - example: zip
- when size of MPI data type and system data type are not equal, the MPI data type can be corrected for portability



MPI collectives: broadcast

- [conway.c](#)